

Wave-12 discovery-pressure audit — captured QA evidence (§11.4.83)

Revision: 2 Last modified: 2026-07-10T04:00:00Z

Run-id: wave12-20260710. Independent adversarial 2nd-pass audits (§11.4.118 discovery-pressure) of already-shipped Go code, each closure carrying real RED → GREEN + conductor-run polarity (§11.4.115) + independent gates (§11.4.125/§11.4.142/§11.4.134). Subagents ran zero git; the conductor is the review+commit seam.

1. server API — two check-then-act TOCTOU races (commit 5e4674c0, main repo)

Audit-trail note: 5e4674c0 was created via `scripts/commit_all.sh -F <file>`, but that wrapper supports only `-m/COMMIT_MESSAGE` (not git's `-F`), so its subject landed as literal `-F` and the detailed message was dropped. The code + tests are correct and pushed 4/4; §11.4.113 forbids `amend+force-push` on an already-published commit, so this file is the recovered full evidence.

Defects (both business-invariant TOCTOU — a store read gates a store write, two separate individually-locked calls, no handler-level atomicity; `go test -race` alone can't see it — only outcome assertions do):

1. `handleCreateDeployment` (POST `/api/v1/deployments`) — `ActiveDeploymentForTarget` (conflict check) and `CreateDeployment` (act) unsynchronized ⇒ N concurrent requests for one `os+target_model+group` all observe “no active deployment” and all create ⇒ multiple simultaneously-active deployments, defeating `endpoints.md` §11.1's 409-conflict invariant.
2. `handleCreateRelease` (POST `/api/v1/releases`) — `LatestRelease` (monotonicity check) and `CreateRelease` (act) unsynchronized ⇒ N

concurrent same-version requests all pass “strictly greater than latest” ⇒ duplicate (os, target_model, version) rows, defeating endpoints.md §10.1.

Fix: two `sync.Mutex` fields on `Server` (`deployMu`, `releaseMu`), each `Lock()` + `defer Unlock()` wrapping the whole check-then-act critical section in its handler. Deadlock-free (each handler takes only its own mutex, never both, never nested, no reentrancy). Low-QPS authenticated admin endpoints, so global serialization is acceptable; `go vet copylocks` confirms `Server` is never copied by value.

Files: `server/internal/api/{server.go, handlers_deployment.go, handlers_release.go} + new handlers_{deployment, release}_concurrency_test.go`.

Regression guards (public-API via `httptest` router; delegating delayed{Check, LatestRelease}Repo wraps the REAL `MemoryRepository`, only adding a post-check `Sleep` to widen the window deterministically — §11.4.27 unit-test allowance, no faked business decision): `TestDeploymentCreateConcurrentNoDuplicateActive`, `TestReleaseCreateConcurrentRejectsDuplicateVersion` — each fires `N=10` barrier-released concurrent authenticated POSTs, asserts exactly one 201, rest 409, exactly one row in store.

Independent gates (conductor): `go build / go vet (incl copylocks) / gofmt clean; go test -race green across ./internal/api ./internal/store ./internal/rollout`; both new tests PASS.

Conductor polarity (§11.4.115), run independently: removing each handler’s lock lines (compile-safe — the mutex field becomes an unused struct field, which Go permits, no §11.4.1 build break) ⇒ the matching test FAILs with `created=10` (TOCTOU reproduced on both); build compiled under both mutations; both source files restored byte-identical (sha256); restored tree GREEN. `ALL_POLARITY_OK=True`. Harness: `scratchpad/polarity_server_toctou.py`.

Published: 5e4674c0 pushed 4/4 (github, gitlab, gitflic, gitverse — all OK).

2. submodules/containers — three concurrency defects (commit f25228f, parent bump 1970c851)

Defects (all fixed): 1. pkg/network/overlay.go — TunnelOverlay.networks map mutated by Create/Delete/Connect/Disconnect + read by List with ZERO synchronization (sole exception among this package's mutex-guarded stateful types) ⇒ genuine data race, can runtime-fatal “concurrent map writes”. Fix: mu sync.Mutex, lock all 5 methods. (Assigned LEAD-1.) 2. pkg/scheduler/scheduler.go — DefaultScheduler.placements written by Schedule/ScheduleBatch while StrategySpread reads it in a sort.Slice comparator ⇒ concurrent map read+write, can runtime-fatal. Fix: mu sync.Mutex around the read(scheduleOne)+write(++) section, I/O left outside the lock. 3. pkg/event/bus.go — Close() held b.mu (deferred) across sub.stop()'s blocking <-s.done; a re-entrant handler (Publish/Subscribe) deadlocks. Fix: collect+delete subs under the lock, release, then stop() outside — mirrors the already-correct Unsubscribe.

Assigned-lead FACT determinations (§11.4.6): LEAD-1 fixed; LEAD-2 (port_allocator end-exclusive range) = NOT a defect (matches the package's own tested contract — exhaustion tests fill [start,end)); LEAD-3 (dotenv inline-# stripping) = ambiguous contract, not reachable via this project's #-free config schema, left unmodified with a maintainer note (no guess-driven change).

Regression guards: overlay_race_test.go (25×3 concurrent Connect/Disconnect/List + post-storm consistent-state assertion), scheduler_race_test.go (25×2 concurrent Schedule/ScheduleBatch under Spread), bus_deadlock_test.go (re-entrant handler Publish at Close time, bounded 2s timeout detects the deadlock without hanging).

Independent gates + polarity (conductor): build/vet/gofmt clean; -race green (3 pkgs); polarity harness scratchpad/polarity_containers_w12.py — M1 remove overlay locks → DATA RACE; M2 remove scheduler locks → DATA RACE; M3 reorder Close() so stop() runs under the lock → DEADLOCK (2.05s timeout); all three compile (mutex fields stay declared → sync import stays used, no §11.4.1 break); all restored byte-identical; restored tree GREEN. ALL_POLARITY_OK=True. **Published:** f25228f FF github+gitlab (ab146b4..f25228f); parent bump 1970c851 pushed 4/4.

3. server device — signature verifier fails closed on a malformed key (main repo)

Defect: `SignatureVerifier.KeyConfigured()` reports true whenever `len(publicKey) > 0`, so a non-empty but wrong-length trusted key (truncated / corrupted `HELIX_ARTIFACT_PUBKEY`) is treated as configured and verification is NOT skipped — but `Verify()` handed it to `crypto/ed25519.Verify`, which PANICS (“bad public key length”) for any length $\neq 32$. A trust-boundary verification primitive must fail CLOSED, never crash. Fix: `len(v.publicKey) != ed25519.PublicKeySize` guard returning a fail-closed error, mirroring the sibling `ota-artifact-validator.ValidateSignature`.

Reachability (§11.4.6 honesty): the verifier is currently test-only — applyport does not yet wire it in — so the panic is not reachable via a live request today; it is a real contract defect in a shipped crypto primitive that fires the moment apply-port wires it. The strictly-in-scope upload trust chain was verified SOLID in the same pass: `resolvePublicKey` takes the key only from config (never the request); S2/S3/S6 bind attacker hash-file/meta.SHA256/ signature to the real computed digest (no forge/replay); the split-signature acceptance hole is already closed; empty/no-key/wrong-len-key/non-base64/ wrong-len-sig all already fail closed there.

Regression guard: `signature_malformed_key_test.go`
`TestSignatureVerifier_Verify_MalformedKeyFailsClosedNoPanic` — 5-byte key, asserts `KeyConfigured()==true`, well-formed correct-length sig, `Verify` in a `recover()` wrapper, `Fatalf` on panic AND on `err==nil`.

Independent gates + polarity (conductor): `build/vet/gofmt` clean; `-race` green; `polarity scratchpad/polarity_sigverify.py` — guard → always-false (< 0 , compile-safe) → `Verify` reaches `ed25519.Verify` → panics (“bad public key length: 5”) → test FAILs; restored byte-identical; GREEN. `ALL_POLARITY_OK=True`. Committed with this evidence doc.